

1 Assignment Overview

What you will implement

1. Byte-pair encoding (BPE) tokenizer
2. Transformer language model (LM)
3. The cross-entropy loss function and the AdamW optimizer
4. The training loop, with support for serializing and loading model and optimizer state

DONE

What you will run

1. Train a BPE tokenizer on the TinyStories dataset.
2. Run your trained tokenizer on the dataset to convert it into a sequence of integer IDs.
3. Train a Transformer LM on the TinyStories dataset.
4. Generate samples and evaluate perplexity using the trained Transformer LM.
5. Train models on OpenWebText and submit your attained perplexities to a leaderboard.

DONE

2 Byte-Pair Encoding (BPE) Tokenizer

2.1 The Unicode Standard

In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

Problem (unicode1): Understanding Unicode (1 point)

- (a) What Unicode character does `chr(0)` return?

Answer: `'\x00'`

- (b) How does this character's string representation `__repr__()` differ from its printed representation?

Answer: The `__repr__()` is designed to be **unambiguous** and is mainly for developers, showing the technical details of the object. On the other hand, the printed representation is meant to be **readable** and user-friendly.

- (c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

Answer: Code below.

```
>>> chr(0)
'\x00'
>>> print(chr(0))

>>> "this is a test" + chr(0) + "string"
'this is a test\x00string'
>>> print("this is a test" + chr(0) + "string")
this is a teststring
```

2.2 Unicode Encodings

- UTF-8: The Efficient Variable-Length Encoder, It uses 1 to 4 bytes per character.
- UTF-16: The Middle Ground, It uses either 2 or 4 bytes per character.
- UTF-32: The Simple Fixed-Length Encoder, It uses exactly 4 bytes for every character, regardless of what it is.

Problem (unicode2): Unicode Encodings (3 point)

- (a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

Answer: `'\x00'`

- (b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

Example: `b'\xe4\xb8\xad'`.

Answer: This function is incorrect because it attempts to decode each byte individually, which fails for any character encoded as a multi-byte sequence, resulting in a `UnicodeDecodeError`.

- (c) Give a two byte sequence that does not decode to any Unicode character(s).

Example: `b'\xff\xff'`.

Answer: This sequence is invalid because the byte `0xFF` is explicitly forbidden in the UTF-8 standard and does not correspond to any valid lead or continuation byte in the encoding scheme.

2.3 Subword Tokenization

Sennrich et al. [2016] propose to use byte-pair encoding (BPE; Gage, 1994), a compression algorithm that iteratively replaces (“merges”) the most frequent pair of bytes with a single, new unused index.

2.4 BPE Tokenizer Training

Vocabulary initialization

Pre-tokenization Use regular expressions to split the sentence into independent punctuation marks and independent items that may include spaces, and improve the merging effect by counting the occurrence frequency of these independent items.

```
>>> PAT = r"'''(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!S)|\s+"""
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', 'll', ' pre', '-', 'tokenize']
```

Compute BPE merges During the BPE merging process, two **principles** must be followed:

- Do not **cross** pre-tokenization boundaries (this is already guaranteed algorithmically);
- Handle issues with a **reproducible conflict resolution strategy**, such as selecting the lexicographically largest word pair to merge when the occurrence counts are the same.

Special tokens

2.5 Experimenting with BPE Tokenizer Training

Parallelizing pre-tokenization Using the provided segmentation code and the `multiprocessing` library, parallelize the tokenization statistics.

Removing special tokens before pre-tokenization

Optimizing the merging step After performing a statistical pass to obtain a tokenization table with frequencies, perform merging only within that tokenization table.

Low-Resource/Downscaling Tip: Profiling

You should use profiling tools like `cProfile` or `scalene` to identify the bottlenecks in your implementation, and focus on optimizing those.

Problem (train_bpe): BPE Tokenizer Training (15 points)

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

input_path: `str` Path to a text file with BPE tokenizer training data.

vocab_size: `int` A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).

special_tokens: `list[str]` A list of strings to add to the vocabulary. These special tokens do not otherwise affect BPE training.

Your BPE training function should return the resulting vocabulary and merges:

vocab: `dict[int, bytes]` The tokenizer vocabulary, a mapping from `int` (token ID in the vocabulary) to `bytes` (token bytes).

merges: `list[tuple[bytes, bytes]]` A list of BPE merges produced from training. Each list item is a `tuple` of `bytes` (`<token1>`, `<token2>`), representing that `<token1>` was merged with `<token2>`. The merges should be ordered by order of creation.

Insights: BPE Tokenizer Training

Python plus Rust: Python is used for rapid prototyping, while Rust handles the actual deployment of the product.

Glitch Tokens: Due to insufficiently strict data cleaning, some meaningless words—namely **Mojibake**—may appear during the BPE training process. Moreover, because of BPE’s **greedy, unsupervised, and non-semantic nature**, a large number of such meaningless word merges occur, turning them into abnormally long and nonsensical words.

Vocabulary Bloat (Dead Tokens): Because BPE is incremental and never removes merges,

the intermediate products of long words permanently occupy the vocabulary, leading to wasted model parameters and tokenization fragmentation. In industry, there are three approaches to address this issue: First, absolutely aggressive data cleaning; second, stricter regex filtering. Both of these are common methods for limiting glitch tokens. The third approach is to switch to the **Unigram** algorithm, which is frequently used by Google-family models (such as T5 and some Llama variants).

Unigram: Unigram works by "subtraction": it first initializes an extremely large vocabulary (e.g., 100,000 words) and **then calculates how much the overall corpus compression rate would be affected if a given word were removed**. Those useless intermediate composite words are directly eliminated and discarded during training.

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

- (a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How many hours and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 30 minutes (no GPUs), ≤ 30 GB RAM

Resource used: 10.78 seconds, 2.6GB RAM

Longest token: 'disappointment'

- (b) Profile your code. What part of the tokenizer training process takes the most time?

Answer: `process_chunk_with_special_tokens` takes up the vast majority of the time.

◇ ◇ ◇

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)

- (a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 12 hours (no GPUs), ≤ 100 GB RAM

Resource used: 6 minutes, 7.0GB RAM

Longest token: '_____'

- (b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Answer: Because TinyStories has an extremely limited vocabulary, a 10k vocab size forces the BPE tokenizer to over-merge, creating tokens that represent entire multi-word phrases. In contrast, OpenWebText's diverse internet text requires the BPE algorithm to

rely heavily on subwords (like prefixes and suffixes) to represent its massive vocabulary within a 32k limit, resulting in shorter, more fragmented tokens for rare words.

2.6 BPE Tokenizer: Encoding and Decoding

2.6.1 Encoding text

Special tokens.

Memory considerations.

2.6.2 Decoding text

Problem (tokenizer): Implementing the tokenizer (15 points)

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren't already there).

Answer: code in appendix.

2.7 Experiments

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

- (a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Answer: The TinyStories tokenizer achieves a higher compression ratio (5.5 bytes/token) because its vocabulary easily memorizes long, frequent whole words from its restricted domain. In contrast, the OpenWebText tokenizer has a lower compression ratio (3.8 bytes/token) as it must rely heavily on shorter subwords to encode a highly diverse vocabulary.

- (b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Answer: Tokenizing OpenWebText with the TinyStories tokenizer causes a severe domain mismatch, forcing the tokenizer to aggressively fragment unfamiliar web text into individual characters or very small byte sequences. Consequently, the token count explodes, resulting in a dramatically lower (worse) compression ratio compared to using the native OpenWebText tokenizer.

- (c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Answer: The throughput of the tokenizer depends on the implementation details, but a rough estimate for a modern CPU might be around 100 MB/second. Given the Pile dataset's size of 825GB, it would take approximately 8.25 hours to tokenize the entire dataset.

- (d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

Answer: `uint16` is an appropriate choice because it can represent all possible token IDs within the range of 0 to 65,535, which is sufficient for most practical applications and allows for efficient memory usage.

3 Transformer Language Model Architecture

3.1 Transformer LM

3.1.1 Token Embeddings

3.1.2 Pre-norm Transformer Block

3.2 Output Normalization and Embedding

3.3 Remark: Batching, Einsum and Efficient Computation

3.3.1 Mathematical Notation and Memory Ordering

3.4 Basic Building Blocks: Linear and Embedding Modules

3.4.1 Parameter Initialization

3.4.2 Linear Module

Problem (linear): Implementing the linear module (1 point)

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch's built-in `nn.Linear` module, except for not having a `bias` argument or parameter. We recommend the following interface:

`def __init__(self, in_features, out_features, device=None, dtype=None)` Construct a linear transformation module. This function should accept the following parameters:

`in_features: int` final dimension of the input

`out_features: int` final dimension of the output

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

`def forward(self, x: torch.Tensor) -> torch.Tensor` Apply the linear transformation to the input.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- construct and store your parameter as W (not W^T) for memory ordering reasons, putting it in an `nn.Parameter`
- of course, don't use `nn.Linear` or `nn.functional.linear`

Code below;

3.4.3 Embedding Module

Problem (embedding): Implement the embedding module (1 point)

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch's built-in `nn.Embedding` module. We recommend the following interface:

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

Construct an embedding module. This function should accept the following parameters:

`num_embeddings`: `int` Size of the vocabulary

`embedding_dim`: `int` Dimension of the embedding vectors, i.e., d_{model}

`device`: `torch.device` | `None` = `None` Device to store the parameters on

`dtype`: `torch.dtype` | `None` = `None` Data type of the parameters

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

Lookup the embedding vectors for the given token IDs.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as a `nn.Parameter`
- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.

To test your implementation, implement the test adapter at `[adapters.run_embedding]`. Then, run `uv run pytest -k test_embedding`.

3.5 Pre-Norm Transformer Block

3.5.1 Root Mean Square Layer Normalization

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Deliverable: Implement `RMSNorm` as a `torch.nn.Module`. We recommend the following interface:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

Construct the `RMSNorm` module. This function should accept the following parameters:

`d_model`: `int` Hidden dimension of the model

```

eps: float = 1e-5 Epsilon value for numerical stability

device: torch.device | None = None Device to store the parameters on

dtype: torch.dtype | None = None Data type of the parameters

```

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.

To test your implementation, implement the test adapter at `[adapters.run_rmsnorm]`. Then, run `uv run pytest -k test_rmsnorm`.

3.5.2 Position-Wise Feed-Forward Network

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

Deliverable: Implement the SwiGLU feed-forward network, composed of a SiLU activation function and a GLU.

Note: in this particular case, you should feel free to use `torch.sigmoid` in your implementation for numerical stability.

You should set d_{ff} to approximately $\frac{8}{3} \times d_{model}$ in your implementation, while ensuring that the dimensionality of the inner feed-forward layer is a multiple of 64 to make good use of your hardware. To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_swiglu]`. Then, run `uv run pytest -k test_swiglu` to test your implementation.

3.5.3 Relative Positional Embeddings

Problem (rope): Implement RoPE (2 points)

Deliverable: Implement a class `RotaryPositionalEmbedding` that applies RoPE to the input tensor. The following interface is recommended:

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

Construct the RoPE module and create buffers if needed.

```

theta: float  $\Theta$  value for the RoPE

d_k: int dimension of query and key vectors

max_seq_len: int Maximum sequence length that will be inputted

device: torch.device | None = None Device to store the buffer on

```

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

Process an input tensor of shape `(..., seq_len, d_k)` and return a tensor of the same shape. Note that you should tolerate `x` with an arbitrary number of batch dimensions. You should assume that the token positions are a tensor of shape `(..., seq_len)` specifying the token positions of `x` along the sequence dimension.

You should use the token positions to slice your (possibly precomputed) `cos` and `sin` tensors along the sequence dimension.

To test your implementation, complete `[adapters.run_rope]` and make sure it passes `uv run pytest -k test_rope`.

3.5.4 Scaled Dot-Product Attention

Problem (softmax): Implement softmax

(1 point)

Deliverable: Write a function to apply the softmax operation on a tensor. Your function should take two parameters: a tensor and a *dimension* `i`, and apply softmax to the `i`-th dimension of the input tensor. The output tensor should have the same shape as the input tensor, but its `i`-th dimension will now have a normalized probability distribution. Use the trick of subtracting the maximum value in the `i`-th dimension from all elements of the `i`-th dimension to avoid numerical stability issues.

To test your implementation, complete `[adapters.run_softmax]` and make sure it passes `uv run pytest -k test_softmax_matches_pytorch`.

◇ ◇ ◇

Problem (scaled_dot_product_attention): Implement scaled dot-product attention **(5 points)**

Deliverable: Implement the scaled dot-product attention function. Your implementation should handle keys and queries of shape `(batch_size, ..., seq_len, d_k)` and values of shape `(batch_size, ..., seq_len, d_v)`, where `...` represents any number of other batch-like dimensions (if provided). The implementation should return an output with the shape `(batch_size, ..., d_v)`. See section 3.3 for a discussion on batch-like dimensions.

Your implementation should also support an optional user-provided boolean mask of shape `(seq_len, seq_len)`. The attention probabilities of positions with a mask value of `True` should collectively sum to 1, and the attention probabilities of positions with a mask value of `False` should be zero.

To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_scaled_dot_product_attention]`.

`uv run pytest -k test_scaled_dot_product_attention` tests your implementation on third-order input tensors, while `uv run pytest -k test_4d_scaled_dot_product_attention`

tests your implementation on fourth-order input tensors.

3.5.5 Causal Multi-Head Self-Attention

Problem (multihead_self_attention):

Implement causal multi-head self-attention (5 points)

Deliverable: Implement causal multi-head self-attention as a `torch.nn.Module`. Your implementation should accept (at least) the following parameters:

`d_model`: `int` Dimensionality of the Transformer block inputs.

`num_heads`: `int` Number of heads to use in multi-head self-attention.

Following Vaswani et al. [2017], set $d_k = d_v = d_{\text{model}}/h$.

To test your implementation against our provided tests, implement the test adapter at `[adapters.run_multihead_self_attention]`. Then, run `uv run pytest -k test_multihead_self_attention` to test your implementation.

3.6 The Full Transformer LM

Problem (transformer_block): Implement the Transformer block (3 points)

Implement the pre-norm Transformer block as described in §3.5 and illustrated in Figure 2. Your Transformer block should accept (at least) the following parameters.

`d_model`: `int` Dimensionality of the Transformer block inputs.

`num_heads`: `int` Number of heads to use in multi-head self-attention.

`d_ff`: `int` Dimensionality of the position-wise feed-forward inner layer.

To test your implementation, implement the adapter `[adapters.run_transformer_block]`. Then run `uv run pytest -k test_transformer_block` to test your implementation.

Deliverable: Transformer block code that passes the provided tests.

◇ ◇ ◇

Problem (transformer_lm): Implementing the Transformer LM (3 points)

Time to put it all together! Implement the Transformer language model as described in §3.1 and illustrated in Figure 1. At minimum, your implementation should accept all the aforementioned construction parameters for the Transformer block, as well as these additional parameters:

`vocab_size`: `int` The size of the vocabulary, necessary for determining the dimensionality of the token embedding matrix.

`context_length`: `int` The maximum context length, necessary for determining the dimen-

sionality of the position embedding matrix.

`num_layers`: `int` The number of Transformer blocks to use.

To test your implementation against our provided tests, you will first need to implement the test adapter at `[adapters.run_transformer_lm]`. Then, run `uv run pytest -k test_transformer_lm` to test your implementation.

Deliverable: A Transformer LM module that passes the above tests.

◇ ◇ ◇

Problem (transformer_accounting): Transformer LM resource accounting (5 points)

Subproblem(a)

Consider GPT-2 XL, which has the following configuration:

```
vocab_size      50,257
context_length  1,024
num_layers      48
d_model         1,600
num_heads       25
d_ff            6,400
```

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Answer:

A. Token Embeddings

- **Calculation:** $\text{vocab_size} \times d_{\text{model}} = 50,257 \times 1,600 = \mathbf{80,411,200}$
- **Function:** Maps discrete word IDs to continuous numerical vectors; this is the first step in the model's understanding of language.

B. Transformer Blocks (48 layers in total)

Each Block contains:

1. Attention Projections:

- Four matrices W_q, W_k, W_v, W_o , each of size $d_{\text{model}} \times d_{\text{model}}$.
- **Parameters:** $4 \times (1,600 \times 1,600) = 10,240,000$

2. SwiGLU Feed-Forward Network (FFN):

- According to your implementation, it contains W_1, W_3 (up-projection) and W_2 (down-projection).
- **Parameters:** $3 \times (d_{\text{model}} \times d_{\text{ff}}) = 3 \times (1,600 \times 6,400) = 30,720,000$

3. Layer Normalization (RMSNorm):

- **ln1 and ln2** each have a learnable scale parameter of length d_{model} .
- **Parameters:** $2 \times 1,600 = 3,200$

Subtotal per layer: $10,240,000 + 30,720,000 + 3,200 = 40,963,200$

Total for 48 layers: $48 \times 40,963,200 = \mathbf{1,966,233,600}$

C. Final Output Layer (Final Norm & LM Head)

- **Final RMSNorm:** 1,600
- **LM Head:** Maps the hidden state back to the vocabulary, $d_{\text{model}} \times \text{vocab_size} = 1,600 \times 50,257 = 80,411,200$
- **Function:** Converts the high-dimensional features learned by the model back into a probability distribution over words, used to predict the next word.

Total Parameters:

$80,411,200 + 1,966,233,600 + 80,411,200 = \mathbf{2,127,056,000}$

Memory Cost(FP32):

$2,127,056,000 \times 4 \text{ Bytes} \approx \mathbf{8,508,230,400 \text{ Bytes}}$

Subproblem(b)

Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has `context_length` tokens.

Answer:

A. The Foyer (Entrance)

- **Token Embedding:** Pure memory lookup to fetch the 1,600-dimensional vector. No arithmetic operations. **FLOPs: 0.**
- **Positional Encoding (RoPE):** Multiplies features by sin and cos. Approx 4 operations per element.
FLOPs: $4 \times s \times d = 4 \times 1024 \times 1600 \approx \mathbf{6.55 \text{ MFLOPs.}}$

B. Transformer Blocks (Single Layer)

Each Block contains:

1. Attention Mechanism:

- **RMSNorm 1:** Approx 5 ops per element. **FLOPs:** $5 \times s \times d \approx 8 \text{ MFLOPs.}$
- **Q, K, V Projections (W_q, W_k, W_v):** Complexity $\mathcal{O}(sd^2)$.
FLOPs: $3 \times 2sd^2 \approx \mathbf{15.73 \text{ GFLOPs.}}$
- **Attention Scores & Value Extraction:** $Q \times K^\top$ and $\text{Score} \times V$.
FLOPs: $2s^2d + 2s^2d \approx \mathbf{6.71 \text{ GFLOPs.}}$

- **Internal Softmax:** Approx 3 ops per element. **FLOPs:** $3 \times s^2 \times h \approx 78.64$ MFLOPs.
- **Output Projection (W_o):** **FLOPs:** $2sd^2 \approx 5.24$ GFLOPs.
- **Residual Add 1:** **FLOPs:** $s \times d \approx 1.63$ MFLOPs.

2. SwiGLU Feed-Forward Network (FFN):

- **RMSNorm 2:** **FLOPs:** ≈ 8 MFLOPs.
- **Up/Gate (W_1, W_3) & Down (W_2) Projections:**
FLOPs: $(2 \times 2sdd_{\text{ff}}) + (2sdd_{\text{ff}}) = 6sdd_{\text{ff}} \approx 62.91$ GFLOPs.
- **SiLU & Gating:** Approx 5 ops per element. **FLOPs:** $5 \times s \times d_{\text{ff}} \approx 32.76$ MFLOPs.
- **Residual Add 2:** **FLOPs:** ≈ 1.63 MFLOPs.

Single Block Summary:

Large Matrix Multiplications: ≈ 90.60 GFLOPs

Minor Operations (Norms/Acts/Adds): ≈ 137.21 MFLOPs (Only $\approx 0.15\%$ of a single layer)

C. Scaling Up (48 Layers)

- **Core Matrix Operations:** 48×90.60 GFLOPs $\approx 4,348.65$ GFLOPs.
- **Minor Operations:** 48×137.21 MFLOPs ≈ 6.58 GFLOPs.

D. The Exit (Final Output)

- **Final RMSNorm:** **FLOPs:** ≈ 8 MFLOPs.
- **LM Head Projection:** Maps 1600D features to vocabulary space.
FLOPs: $2 \times s \times d \times V = 2 \times 1024 \times 1600 \times 50,257 \approx 164.68$ GFLOPs.
- **Final Softmax:** Approx 3 ops per element.
FLOPs: $3 \times 1024 \times 50,257 \approx 154.38$ MFLOPs.

Total FLOPs (Forward Pass):

$4348.65 + 6.58 + 164.68 + 0.15 \approx 4,520$ GFLOPs (approx. **4.52** TFLOPs)

Subproblem(c)

Based on your analysis above, which parts of the model require the most FLOPs?

Answer:

Based on the FLOPs analysis, the **Feed-Forward Network (FFN)** requires the absolute majority of the computation during a standard forward pass, rather than the Attention mechanism.

Key Insights:

1. The True Computational Bottleneck is the FFN, not Attention:

Due to the massive matrix dimension expansion (typically $d \rightarrow 4d$, or scaling to d_{ff} in SwiGLU), the computational cost of the FFN is nearly 3 times that of the attention linear

projections (Q, K, V, O). This is precisely why cutting-edge large language models heavily research **Mixture of Experts (MoE)** architectures—sparsifying the FFN activations instantly saves a massive amount of compute!

2. The $\mathcal{O}(s^2)$ Complexity is Only Dominant at Long Contexts:

At a standard sequence length of $s = 1024$, the quadratic attention operations (like $QK^\top$) actually account for a negligible fraction of the total computation. It is only when the sequence length s scales up to tens or hundreds of thousands of tokens that this $\mathcal{O}(s^2)$ bottleneck overtakes the FFN to become the primary computational constraint.

Subproblem(d)

Repeat your analysis with GPT-2 small (12 layers, 768 `d_model`, 12 heads), GPT-2 medium (24 layers, 1024 `d_model`, 16 heads), and GPT-2 large (36 layers, 1280 `d_model`, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Deliverable: For each model, provide a breakdown of model components and its associated FLOPs (as a proportion of the total FLOPs required for a forward pass). In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

Answer:

Assuming the standard sequence length $s = 1024$, vocabulary size $V = 50,257$, and $d_{\text{ff}} = 4 \times d_{\text{model}}$, we calculate the core FLOPs (ignoring minor operations like Norms/Residuals which account for $< 0.2\%$ of compute).

1. GPT-2 Small ($L = 12, d_{\text{model}} = 768$)

- **Total FLOPs:** ≈ 349.6 GFLOPs
- **Attention (Proj + Scores):** ≈ 96.7 GFLOPs (**27.7%**)
- **FFN (SwiGLU):** ≈ 173.9 GFLOPs (**49.7%**)
- **LM Head:** ≈ 79.0 GFLOPs (**22.6%**)

2. GPT-2 Medium ($L = 24, d_{\text{model}} = 1024$)

- **Total FLOPs:** $\approx 1,033.1$ GFLOPs
- **Attention (Proj + Scores):** ≈ 309.2 GFLOPs (**29.9%**)
- **FFN (SwiGLU):** ≈ 618.5 GFLOPs (**59.9%**)
- **LM Head:** ≈ 105.4 GFLOPs (**10.2%**)

3. GPT-2 Large ($L = 36, d_{\text{model}} = 1280$)

- **Total FLOPs:** $\approx 2,257.8$ GFLOPs
- **Attention (Proj + Scores):** ≈ 676.5 GFLOPs (**30.0%**)

- **FFN (SwiGLU):** $\approx 1,449.6$ GFLOPs (**64.2%**)
- **LM Head:** ≈ 131.7 GFLOPs (**5.8%**)

Trend Description:

As the model size increases, the computational cost is heavily dominated by the internal Transformer blocks (specifically the FFN and Attention projections, which scale quadratically with the hidden dimension, $\mathcal{O}(L \cdot s \cdot d^2)$). Consequently, the LM Head (which scales only linearly with the hidden dimension, $\mathcal{O}(s \cdot d \cdot V)$) takes up a proportionally smaller fraction of the total FLOPs, plummeting from nearly 23% in the Small model to roughly 6% in the Large model.

Subproblem(e)

Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How do the relative contribution of FLOPs of the model components change?

Answer:

Increasing the context length s from 1,024 to 16,384 (a $16\times$ increase) causes the total FLOPs to skyrocket from roughly 4.5 TFLOPs to ≈ 149.5 **TFLOPs** (an approximate $33\times$ increase). The relative contributions of the model components change fundamentally: the quadratic $\mathcal{O}(s^2d)$ attention score computation overtakes the FFN to become the absolute computational bottleneck.

Detailed Breakdown:

- **Linear Scaling Components ($\mathcal{O}(s)$, scaled by $16\times$):**
 - **FFN:** ≈ 1006.6 GFLOPs per layer $\times 48 \approx$ **48.3 TFLOPs (32.3%)**
 - **Linear Projections (Q, K, V, O):** ≈ 335.5 GFLOPs per layer $\times 48 \approx$ **16.1 TFLOPs (10.8%)**
 - **LM Head:** $\approx$ **2.6 TFLOPs (1.8%)**
- **Quadratic Scaling Component ($\mathcal{O}(s^2)$, scaled by $256\times$):**
 - **Attention Scores ($QK^\top$ & SV):** The original 6.71 GFLOPs per layer explodes to $\approx 1,718$ GFLOPs per layer!
Total across 48 layers: $\approx$ **82.5 TFLOPs (55.1%)**

Conclusion: At $s = 16,384$, the attention mechanism alone accounts for over **65%** of the total compute (Projections + Scores), completely dwarfing the FFN. This demonstrates why techniques like FlashAttention, Ring Attention, or Sparse Attention are strictly required for long-context language models.

A Complete Source Code

A.1 Fast BPE Implementation (Rust)

```

1  use pyo3::prelude::*;
2  use std::collections::{HashMap, HashSet, BinaryHeap};
3  use std::cmp::Ordering;
4
5  #[derive(Eq, PartialEq)]
6  struct HeapItem {
7      count: usize,
8      pair: (u32, u32),
9  }
10
11  impl Ord for HeapItem {
12      fn cmp(&self, other: &Self) -> Ordering {
13          self.count.cmp(&other.count)
14              .then_with(|| self.pair.cmp(&other.pair))
15      }
16  }
17
18  impl PartialOrd for HeapItem {
19      fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
20          Some(self.cmp(other))
21      }
22  }
23
24  #[pyfunction]
25  fn train_bpe_rust(
26      global_counts: HashMap<Vec<u8>, usize>,
27      target_vocab_size: usize,
28  ) -> PyResult<(Vec<(Vec<u8>, Vec<u8>>), HashMap<usize, Vec<u8>>>) {
29
30      let mut vocab: HashMap<u32, Vec<u8>> = (0..256)
31          .map(|i| (i as u32, vec![i as u8]))
32          .collect();
33      let mut next_id = 256u32;
34      let num_merges = target_vocab_size.saturating_sub(256);
35      let mut merges = Vec::with_capacity(num_merges);
36
37      let mut words: Vec<Vec<u32>> = Vec::new();
38      let mut word_freqs: Vec<usize> = Vec::new();
39
40      for (bytes, freq) in global_counts {
41          words.push(bytes.into_iter().map(|b| b as u32).collect());
42          word_freqs.push(freq);
43      }
44
45      let mut pair_counts: HashMap<(u32, u32), usize> = HashMap::new();
46      let mut pair_to_words: HashMap<(u32, u32), HashSet<usize>> = HashMap::new();
47
48      for (idx, word_ids) in words.iter().enumerate() {
49          let freq = word_freqs[idx];
50          for pair in word_ids.windows(2) {
51              let p = (pair[0], pair[1]);
52              *pair_counts.entry(p).or_insert(0) += freq;
53              pair_to_words.entry(p).or_insert_with(HashSet::new).insert(idx);
54          }

```

```

55     }
56
57     let mut heap: BinaryHeap<HeapItem> = pair_counts.iter()
58         .map(|(&pair, &count)| HeapItem { count, pair })
59         .collect();
60
61     for _ in 0..num_merges {
62         let mut best_pair = None;
63         while let Some(top) = heap.pop() {
64             if let Some(&current_count) = pair_counts.get(&top.pair) {
65                 if current_count == top.count {
66                     best_pair = Some(top.pair);
67                     break;
68                 }
69             }
70         }
71
72         let p = match best_pair {
73             Some(p) => p,
74             None => break,
75         };
76
77         let p1_bytes = vocab.get(&p.0).unwrap().clone();
78         let p2_bytes = vocab.get(&p.1).unwrap().clone();
79         let mut new_token_bytes = p1_bytes;
80         new_token_bytes.extend(p2_bytes);
81
82         merges.push((vocab.get(&p.0).unwrap().clone(), vocab.get(&p.1).unwrap().clone()));
83         vocab.insert(next_id, new_token_bytes);
84
85         let new_id = next_id;
86         next_id += 1;
87
88         if let Some(word_indices) = pair_to_words.remove(&p) {
89             pair_counts.remove(&p);
90
91             for &word_idx in &word_indices {
92                 let freq = word_freqs[word_idx];
93                 let old_ids = &words[word_idx];
94                 let mut new_ids = Vec::new();
95
96                 let mut i = 0;
97                 while i < old_ids.len() {
98                     if i < old_ids.len() - 1 && old_ids[i] == p.0 && old_ids[i+1] == p.1 {
99                         new_ids.push(new_id);
100                         i += 2;
101                     } else {
102                         new_ids.push(old_ids[i]);
103                         i += 1;
104                     }
105                 }
106
107                 for pair in old_ids.windows(2) {
108                     let old_p = (pair[0], pair[1]);
109                     if let Some(count) = pair_counts.get_mut(&old_p) {
110                         *count -= freq;
111                     }
112                 }
113             }

```

```

114         words[word_idx] = new_ids;
115         let updated_ids = &words[word_idx];
116         for pair in updated_ids.windows(2) {
117             let new_p = (pair[0], pair[1]);
118             let count = pair_counts.entry(new_p).or_insert(0);
119             *count += freq;
120             pair_to_words.entry(new_p).or_insert_with(HashSet::new).insert(word_idx);
121             heap.push(HeapItem { count: *count, pair: new_p });
122         }
123     }
124 }
125 }
126
127 let final_vocab = vocab.into_iter().map(|(k, v)| (k as usize, v)).collect();
128 Ok((merges, final_vocab))
129 }
130
131 #[pymodule]
132 fn fast_bpe(m: &Bound<'_, pyo3::types::PyModule>) -> PyResult<()> {
133     m.add_function(wrap_pyfunction!(train_bpe_rust, m)?);
134     Ok(())
135 }

```

A.2 Python BPE Trainer

```

1  import os
2  import time
3  import pickle
4  import regex as re
5  import multiprocessing
6  from typing import BinaryIO
7  from collections import Counter
8
9  import fast_bpe
10
11  # =====
12  # 1. 全局配置与正则定义
13  # =====
14
15  PATTERN = r"('?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!S)\s+"
16  PRETOKEN_REGEX = re.compile(PATTERN)
17
18  # =====
19  # 2. 多进程预分词辅助函数 (保持原样, Python 的强项)
20  # =====
21
22  def process_chunk_with_special_tokens(chunk_text: str, special_tokens: list[str], pretoken_regex: re.Pattern) ->
↳ Counter:
23      split_pattern = "|".join(map(re.escape, special_tokens)) if special_tokens else "(?!)"
24      text_segments = re.split(split_pattern, chunk_text)
25      chunk_counts = Counter()
26      for segment in text_segments:
27          if not segment: continue
28          for match in pretoken_regex.finditer(segment):
29              pre_token = match.group()
30              byte_token = pre_token.encode("utf-8")
31              chunk_counts[byte_token] += 1
32      return chunk_counts
33
34  def find_chunk_boundaries(file: BinaryIO, desired_num_chunks: int, split_special_token: bytes) -> list[int]:
35      if not split_special_token:
36          file.seek(0, os.SEEK_END)
37          file_size = file.tell()
38          chunk_size = file_size // desired_num_chunks
39          boundaries = [i * chunk_size for i in range(desired_num_chunks + 1)]
40          boundaries[-1] = file_size
41          return sorted(set(boundaries))
42
43      assert isinstance(split_special_token, bytes)
44      file.seek(0, os.SEEK_END)
45      file_size = file.tell()
46      file.seek(0)
47      chunk_size = file_size // desired_num_chunks
48      chunk_boundaries = [i * chunk_size for i in range(desired_num_chunks + 1)]
49      chunk_boundaries[-1] = file_size
50      mini_chunk_size = 4096
51      for bi in range(1, len(chunk_boundaries) - 1):
52          initial_position = chunk_boundaries[bi]
53          file.seek(initial_position)
54          while True:
55              mini_chunk = file.read(mini_chunk_size)
56              if mini_chunk == b"":

```

```

57         chunk_boundaries[bi] = file_size
58         break
59     found_at = mini_chunk.find(split_special_token)
60     if found_at != -1:
61         chunk_boundaries[bi] = initial_position + found_at
62         break
63     initial_position += mini_chunk_size
64     return sorted(set(chunk_boundaries))
65
66 def worker_task(file_path: str, start: int, end: int, special_tokens: list[str]) -> Counter:
67     with open(file_path, "rb") as f:
68         f.seek(start)
69         chunk_bytes = f.read(end - start)
70         chunk_text = chunk_bytes.decode("utf-8", errors="ignore")
71         return process_chunk_with_special_tokens(chunk_text, special_tokens, PRETOKEN_REGEX)
72
73 def run_parallel_pretokenization(input_path: str, special_tokens: list[str]) -> Counter:
74     num_workers = multiprocessing.cpu_count()
75     num_tasks = num_workers * 4
76     boundary_token = special_tokens[0].encode('utf-8') if special_tokens else b""
77     with open(input_path, "rb") as f:
78         boundaries = find_chunk_boundaries(f, num_tasks, boundary_token)
79     with multiprocessing.Pool(processes=num_workers) as pool:
80         tasks = [(input_path, start, end, special_tokens) for start, end in zip(boundaries[:-1], boundaries[1:])]
81         partial_counts = pool.starmap(worker_task, tasks)
82     global_counts = Counter()
83     for local_count in partial_counts:
84         global_counts.update(local_count)
85     return global_counts
86
87
88 # =====
89 # 3. 核心大作业交付函数: train_bpe
90 # =====
91
92 def train_bpe(input_path: str, vocab_size: int, special_tokens: list[str]) -> tuple[dict[int, bytes],
93 ↪ list[tuple[bytes, bytes]]]:
94     print(" 开始预分词与频率统计 (Python 多进程)...")
95     global_counts = run_parallel_pretokenization(input_path, special_tokens)
96
97     num_merges = vocab_size - 256 - len(special_tokens)
98     if num_merges < 0:
99         raise ValueError("vocab_size is too small to fit base bytes and special tokens.")
100
101     print(f" 将数据移交 Rust 底层进行合并 (目标总词表大小: {vocab_size - len(special_tokens)})...")
102
103     merges, vocab = fast_bpe.train_bpe_rust(global_counts, vocab_size - len(special_tokens))
104
105     print("Rust 计算完成, 添加 Special Tokens...")
106
107     vocab_idx = 256 + len(merges)
108     for st in special_tokens:
109         vocab[vocab_idx] = st.encode("utf-8")
110         vocab_idx += 1
111
112     print("BPE 训练彻底完成!")
113     return vocab, merges
114

```

```

115 # =====
116 # 测试/执行入口
117 # =====
118 if __name__ == "__main__":
119
120     file_path = "/home/inferior/Projects/CS336/data/owt_train.txt"
121     # file_path = "/home/inferior/Projects/CS336/data/TinyStoriesV2-GPT4-train.txt"
122     target_vocab_size = 32000
123     # target_vocab_size = 10000
124     special_tks = ["<|endoftext|>"]
125
126     print("--- 开始 BPE 训练任务 ---")
127     start_time = time.time()
128
129     vocab, merges = train_bpe(
130         input_path=file_path,
131         vocab_size=target_vocab_size,
132         special_tokens=special_tks
133     )
134
135     end_time = time.time()
136     print(f"\n[Done] 训练总耗时: {end_time - start_time:.2f} 秒")
137     print(f" 最终合并列表长度: {len(merges)}")
138     print(f" 最终词表大小: {len(vocab)}")
139
140     # -----
141     # 序列化保存 (Serialization)
142     # -----
143     output_dir = "tokenizer_outputs"
144     os.makedirs(output_dir, exist_ok=True)
145
146     vocab_path = os.path.join(output_dir, "vocab.pkl")
147     merges_path = os.path.join(output_dir, "merges.pkl")
148
149     with open(vocab_path, "wb") as f:
150         pickle.dump(vocab, f)
151
152     with open(merges_path, "wb") as f:
153         pickle.dump(merges, f)
154
155     print(f"[Saved] 序列化完成! 文件已保存至: {output_dir}/")
156
157     # -----
158     # 分析: 寻找最长的 Token (作业要求)
159     # -----
160     longest_token_id = max(vocab, key=lambda k: len(vocab[k]))
161     longest_token_bytes = vocab[longest_token_id]
162
163     print("\n[Inspect] 词表分析:")
164     print(f" 最长 Token 的 ID: {longest_token_id}")
165     print(f" 最长 Token 的字节长度: {len(longest_token_bytes)}")
166     try:
167         print(f" 最长 Token 的文本内容: {longest_token_bytes.decode('utf-8')!r}")
168     except UnicodeDecodeError:
169         print(f" 最长 Token (原始字节): {longest_token_bytes}")
170
171     print("\n最初的 5 次 Merge:")
172     for i in range(min(5, len(merges))):
173         p1, p2 = merges[i]

```

```
174     print(f" {p1} + {p2} -> {p1 + p2}")
```

A.3 Tokenizer

```

1  import pickle
2  import regex as re
3  from typing import Iterable, Iterator
4
5  # =====
6  # 1. 全局配置与正则定义
7  # =====
8  PATTERN = r"''(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!\\S)\\s+"
9  PRETOKEN_REGEX = re.compile(PATTERN)
10
11 # =====
12 # 2. Tokenizer 核心类
13 # =====
14 class Tokenizer:
15     def __init__(self, vocab: dict[int, bytes], merges: list[tuple[bytes, bytes]], special_tokens: list[str] |
        ↳ None = None):
16         """
17         初始化 Tokenizer, 建立查找索引并处理特殊 Token。
18         """
19         self.vocab = vocab.copy()
20         self.merges = merges
21         self.special_tokens = special_tokens or []
22
23         # 建立反向查找字典, 用于快速获取 token 对应的 ID
24         self.token_to_id = {v: k for k, v in self.vocab.items()}
25
26         # 将 merges 列表转化为 {pair: rank} 字典, 以 O(1) 的时间复杂度获取合并优先级
27         self.merge_ranks = {pair: i for i, pair in enumerate(self.merges)}
28
29         # 动态处理传入的 special_tokens: 如果不在词表中, 则追加到末尾
30         next_id = max(self.vocab.keys()) + 1 if self.vocab else 0
31         for st in self.special_tokens:
32             st_bytes = st.encode("utf-8")
33             if st_bytes not in self.token_to_id:
34                 self.vocab[next_id] = st_bytes
35                 self.token_to_id[st_bytes] = next_id
36                 next_id += 1
37
38         # 编译用于切分特殊 Token 的正则表达式
39         self.st_pattern = None
40         if self.special_tokens:
41             # 使用捕获组 (...), 这样 re.split 时能保留特殊 token 在列表中
42             sorted_st = sorted(self.special_tokens, key=len, reverse=True)
43             pattern_str = "|".join(map(re.escape, sorted_st))
44             self.st_pattern = re.compile(f"({pattern_str})")
45
46     @classmethod
47     def from_files(cls, vocab_filepath: str, merges_filepath: str, special_tokens: list[str] | None = None):
48         """
49         从序列化的 pickle 文件中加载 vocab 和 merges 并实例化 Tokenizer。
50         """
51         with open(vocab_filepath, "rb") as f:
52             vocab = pickle.load(f)
53         with open(merges_filepath, "rb") as f:
54             merges = pickle.load(f)
55
56         return cls(vocab, merges, special_tokens)

```

```

57
58 def _bpe_encode_chunk(self, chunk_text: str) -> list[int]:
59     """
60     内部辅助函数：对单个正则切割后的纯文本 pre-token 进行 BPE 合并操作。
61     """
62     # 将字符串打散成单字节列表
63     b_list = [bytes([b]) for b in chunk_text.encode("utf-8")]
64
65     # 如果长度为 1，直接返回（基础 256 字节必然在词表中）
66     if len(b_list) <= 1:
67         return [self.token_to_id[b] for b in b_list]
68
69     while True:
70         # 找到当前序列中，在 merges 中优先级最高（rank 最小）的相邻字节对
71         min_rank = float('inf')
72         best_pair = None
73
74         for i in range(len(b_list) - 1):
75             pair = (b_list[i], b_list[i+1])
76             rank = self.merge_ranks.get(pair)
77             if rank is not None and rank < min_rank:
78                 min_rank = rank
79                 best_pair = pair
80
81         # 如果没有任何一对在 merge_ranks 中，说明合并彻底完成
82         if best_pair is None:
83             break
84
85         # 执行最高优先级的一轮合并
86         new_b_list = []
87         i = 0
88         while i < len(b_list):
89             # 命中目标 pair
90             if i < len(b_list) - 1 and b_list[i] == best_pair[0] and b_list[i+1] == best_pair[1]:
91                 new_b_list.append(best_pair[0] + best_pair[1])
92                 i += 2
93             else:
94                 new_b_list.append(b_list[i])
95                 i += 1
96
97         b_list = new_b_list
98
99         # 如果只剩下一个完整的 token，提前退出
100         if len(b_list) == 1:
101             break
102
103         # 拿着最终合并好的字节序列，去词表里换取对应的 IDs
104         return [self.token_to_id[b] for b in b_list]
105
106 def encode(self, text: str) -> list[int]:
107     """
108     将整段输入文本编码为 token IDs。
109     """
110     ids = []
111
112     # 1. 全局截断：优先将文本通过 special_tokens 切开
113     if self.st_pattern:
114         parts = self.st_pattern.split(text)
115     else:

```

```

116         parts = [text]
117
118     for part in parts:
119         if not part:
120             continue
121
122         # 2. 如果这块 part 是一个 special token, 直接查 ID
123         if part in self.special_tokens:
124             ids.append(self.token_to_id[part.encode("utf-8")])
125         else:
126             # 3. 对普通文本进行预分词匹配
127             for match in PRETOKEN_REGEX.finditer(part):
128                 chunk_text = match.group()
129                 ids.extend(self._bpe_encode_chunk(chunk_text))
130
131     return ids
132
133 def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]:
134     """
135     懒加载生成器：逐块/逐行读取流数据并输出 IDs, 确保内存开销维持在常数级别。
136     说明：对于标准的文件句柄 (File Handle), 每次 yield 的是一行。
137     换行符属于 BPE 空格家族, 因此按行拆分不会截断合法的词组边界。
138     """
139     for chunk in iterable:
140         yield from self.encode(chunk)
141
142 def decode(self, ids: list[int]) -> str:
143     """
144     将整数 ID 序列还原为字符串。非法字节将被自动替换为 U+FFFD ()。
145     """
146     raw_bytes = b"".join(self.vocab[idx] for idx in ids)
147     return raw_bytes.decode("utf-8", errors="replace")

```

A.4 Adapters

```

1  from __future__ import annotations
2
3  import os
4  import math
5  import collections
6  import json
7  import regex
8  from collections.abc import Iterable
9  from typing import IO, Any, BinaryIO, Dict, List, Tuple
10
11 import numpy.typing as npt
12 import torch
13 import torch.nn.functional as F
14 import einx
15 from jaxtyping import Bool, Float, Int
16 from torch import Tensor
17 from collections import defaultdict
18 import pickle
19
20 from cs336_basics.test_function.bpe_trainer import train_bpe
21 from cs336_basics.test_function.tokenizer import Tokenizer
22 from cs336_basics.Chap_2.linear import Linear
23 from cs336_basics.Chap_2.embedding import Embedding
24 from cs336_basics.Chap_2.rmsnorm import RMSNorm
25 from cs336_basics.Chap_2.swiglu import SwiGLU
26 from cs336_basics.Chap_2.rope import RotaryPositionalEmbedding
27 from cs336_basics.Chap_2.attention import MultiHeadAttention
28 from cs336_basics.Chap_2.transformer import TransformerBlock
29 from cs336_basics.Chap_2.language_model import TransformerLM
30
31
32 def run_linear(
33     d_in: int,
34     d_out: int,
35     weights: Float[Tensor, " d_out d_in"],
36     in_features: Float[Tensor, " ... d_in"],
37 ) -> Float[Tensor, " ... d_out"]:
38     """
39     Given the weights of a Linear layer, compute the transformation of a batched input.
40
41     Args:
42         in_dim (int): The size of the input dimension
43         out_dim (int): The size of the output dimension
44         weights (Float[Tensor, "d_out d_in"]): The linear weights to use
45         in_features (Float[Tensor, "... d_in"]): The output tensor to apply the function to
46
47     Returns:
48         Float[Tensor, "... d_out"]: The transformed output of your linear module.
49     """
50
51     model = Linear(
52         in_features=d_in,
53         out_features=d_out,
54         device=in_features.device,
55         dtype=in_features.dtype,
56     )
57

```

```

58     model.load_state_dict({"W": weights}, strict=True)
59
60     model.eval()
61     with torch.no_grad():
62         output = model(in_features)
63
64     return output
65
66
67 def run_embedding(
68     vocab_size: int,
69     d_model: int,
70     weights: Float[Tensor, " vocab_size d_model"],
71     token_ids: Int[Tensor, " ..."],
72 ) -> Float[Tensor, " ... d_model"]:
73     """
74     Given the weights of an Embedding layer, get the embeddings for a batch of token ids.
75
76     Args:
77         vocab_size (int): The number of embeddings in the vocabulary
78         d_model (int): The size of the embedding dimension
79         weights (Float[Tensor, "vocab_size d_model"]): The embedding vectors to fetch from
80         token_ids (Int[Tensor, "..."]): The set of token ids to fetch from the Embedding layer
81
82     Returns:
83         Float[Tensor, "... d_model"]: Batch of embeddings returned by your Embedding layer.
84     """
85
86     model = Embedding(
87         num_embeddings=vocab_size,
88         embedding_dim=d_model,
89         device=weights.device,
90         dtype=weights.dtype
91     )
92
93     model.load_state_dict({"weight": weights}, strict=True)
94
95     model.eval()
96     with torch.no_grad():
97         output = model(token_ids)
98
99     return output
100
101
102 def run_swiglu(
103     d_model: int,
104     d_ff: int,
105     w1_weight: Float[Tensor, " d_ff d_model"],
106     w2_weight: Float[Tensor, " d_model d_ff"],
107     w3_weight: Float[Tensor, " d_ff d_model"],
108     in_features: Float[Tensor, " ... d_model"],
109 ) -> Float[Tensor, " ... d_model"]:
110     """Given the weights of a SwiGLU network, return
111     the output of your implementation with these weights.
112
113     Args:
114         d_model (int): Dimensionality of the feedforward input and output.
115         d_ff (int): Dimensionality of the up-project happening internally to your swiglu.
116         w1_weight (Float[Tensor, "d_ff d_model"]): Stored weights for W1

```

```

117         w2_weight (Float[Tensor, "d_model d_ff"]): Stored weights for W2
118         w3_weight (Float[Tensor, "d_ff d_model"]): Stored weights for W3
119         in_features (Float[Tensor, "... d_model"]): Input embeddings to the feed-forward layer.
120
121     Returns:
122         Float[Tensor, "... d_model"]: Output embeddings of the same shape as the input embeddings.
123     """
124     # Example:
125     # If your state dict keys match, you can use `load_state_dict()`
126     # swiglu.load_state_dict(weights)
127     # You can also manually assign the weights
128     # swiglu.w1.weight.data = w1_weight
129     # swiglu.w2.weight.data = w2_weight
130     # swiglu.w3.weight.data = w3_weight
131     model = SwiGLU(
132         d_model=d_model,
133         d_ff=d_ff,
134         device=in_features.device,
135         dtype=in_features.dtype
136     )
137
138     state_dict = {
139         "w1.W": w1_weight,
140         "w2.W": w2_weight,
141         "w3.W": w3_weight
142     }
143     model.load_state_dict(state_dict, strict=True)
144
145     model.eval()
146     with torch.no_grad():
147         output = model(in_features)
148
149     return output
150
151
152 def run_scaled_dot_product_attention(
153     Q: Float[Tensor, "... queries d_k"],
154     K: Float[Tensor, "... keys d_k"],
155     V: Float[Tensor, "... values d_v"],
156     mask: Bool[Tensor, "... queries keys"] | None = None,
157 ) -> Float[Tensor, "... queries d_v"]:
158     """
159     Given key (K), query (Q), and value (V) tensors, return
160     the output of your scaled dot product attention implementation.
161
162     Args:
163         Q (Float[Tensor, "... queries d_k"]): Query tensor
164         K (Float[Tensor, "... keys d_k"]): Key tensor
165         V (Float[Tensor, "... values d_v"]): Values tensor
166         mask (Bool[Tensor, "... queries keys"] | None): Mask tensor
167
168     Returns:
169         Float[Tensor, "... queries d_v"]: Output of SDPA
170     """
171     # 提取 d_k 的维度大小, 用于后续的缩放
172     d_k = Q.size(-1)
173
174     # 1. 计算打分矩阵并缩放: (Q @ K^T) / sqrt(d_k)
175     # K.transpose(-2, -1) 的魔法: 只反转最后两个维度 (keys 和 d_k),
176     # 完全无视前面有多少个 batch 维度 (...), 极其优雅安全!

```

```

176     # 结果 scores 的形状将会是: (... queries, keys)
177     scores = (Q @ K.transpose(-2, -1)) / math.sqrt(d_k)
178
179     # 2. 应用掩码 (Masking)
180     if mask is not None:
181         # mask 中值为 True 表示“允许看”，False 表示“遮挡不允许看”。
182         # ~mask 取反后，把所有不允许看的位置强制填充为 -inf
183         scores = scores.masked_fill(~mask, float('-inf'))
184
185     # 3. Softmax 归一化
186     # 沿着最后一个维度 (keys) 将得分转化为概率分布。
187     # 这里你可以调用上一题自己手写的 run_softmax(scores, dim=-1),
188     # 但为了保证在多维 Tensor 上的极致性能，直接调用内置的 torch.softmax 也是绝佳选择。
189     attention_weights = torch.softmax(scores, dim=-1)
190
191     # 4. 乘以 V (提取信息)
192     # attention_weights 形状: (... queries, keys)
193     # V 形状: (... keys, d_v)
194     # 矩阵乘法后得到最终结果形状: (... queries, d_v)
195     output = attention_weights @ V
196
197     return output
198
199
200 def run_multihead_self_attention(
201     d_model: int,
202     num_heads: int,
203     q_proj_weight: Float[Tensor, "d_k d_in"],
204     k_proj_weight: Float[Tensor, "d_k d_in"],
205     v_proj_weight: Float[Tensor, "d_v d_in"],
206     o_proj_weight: Float[Tensor, "d_model d_v"],
207     in_features: Float[Tensor, "... sequence_length d_in"],
208 ) -> Float[Tensor, "... sequence_length d_out"]:
209     """
210     Given the key, query, and value projection weights of a naive unbatched
211     implementation of multi-head attention, return the output of an optimized batched
212     implementation. This implementation should handle the key, query, and value projections
213     for all heads in a single matrix multiply.
214     This function should not use RoPE.
215     See section 3.2.2 of Vaswani et al., 2017.
216
217     Args:
218         d_model (int): Dimensionality of the feedforward input and output.
219         num_heads (int): Number of heads to use in multi-headed attention.
220         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
221         q_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the Q projection
222         k_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the K projection
223         v_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the V projection
224         o_proj_weight (Float[Tensor, "d_model d_v"]): Weights for the output projection
225         in_features (Float[Tensor, "... sequence_length d_in"]): Tensor to run your implementation on.
226
227     Returns:
228         Float[Tensor, "... sequence_length d_out"]: Tensor with the output of running your optimized, batched
229         ↪ multi-headed attention
230         implementation with the given QKV projection weights and input features.
231     """
232     # 1. 实例化
233     model = MultiHeadAttention(
234         d_model=d_model,

```

```

234         num_heads=num_heads,
235         device=in_features.device,
236         dtype=in_features.dtype
237     )
238
239     # 2. 加载权重 (由于我们的 Linear 类内部参数叫 W, 所以加 .W 后缀)
240     state_dict = {
241         "q_proj.W": q_proj_weight,
242         "k_proj.W": k_proj_weight,
243         "v_proj.W": v_proj_weight,
244         "o_proj.W": o_proj_weight
245     }
246     model.load_state_dict(state_dict, strict=True)
247
248     # 3. 执行推理
249     model.eval()
250     with torch.no_grad():
251         output = model(in_features)
252
253     return output
254
255
256 def run_multihead_self_attention_with_rope(
257     d_model: int,
258     num_heads: int,
259     max_seq_len: int,
260     theta: float,
261     q_proj_weight: Float[Tensor, "d_k d_in"],
262     k_proj_weight: Float[Tensor, "d_k d_in"],
263     v_proj_weight: Float[Tensor, "d_v d_in"],
264     o_proj_weight: Float[Tensor, "d_model d_v"],
265     in_features: Float[Tensor, "... sequence_length d_in"],
266     token_positions: Int[Tensor, "... sequence_length"] | None = None,
267 ) -> Float[Tensor, "... sequence_length d_out"]:
268     """
269     Given the key, query, and value projection weights of a naive unbatched
270     implementation of multi-head attention, return the output of an optimized batched
271     implementation. This implementation should handle the key, query, and value projections
272     for all heads in a single matrix multiply.
273     This version of MHA should include RoPE.
274     In this case, the RoPE embedding dimension must be the head embedding dimension (d_model // num_heads).
275     See section 3.2.2 of Vaswani et al., 2017.
276
277     Args:
278         d_model (int): Dimensionality of the feedforward input and output.
279         num_heads (int): Number of heads to use in multi-headed attention.
280         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
281         theta (float): RoPE parameter.
282         q_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the Q projection
283         k_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the K projection
284         v_proj_weight (Float[Tensor, "d_k d_in"]): Weights for the V projection
285         o_proj_weight (Float[Tensor, "d_model d_v"]): Weights for the output projection
286         in_features (Float[Tensor, "... sequence_length d_in"]): Tensor to run your implementation on.
287         token_positions (Int[Tensor, "... sequence_length"] | None): Optional tensor with the positions of the
288             ↪ tokens
289
290     Returns:
291         Float[Tensor, "... sequence_length d_out"]: Tensor with the output of running your optimized, batched
292             ↪ multi-headed attention

```



```

291         implementation with the given QKV projection weights and input features.
292         """
293         # 1. 维度准备
294         d_head = d_model // num_heads
295         seq_len = in_features.shape[-2]
296
297         # 2. 线性投影与多头切分 (一步位移)
298         # 直接投影后切成 [... heads, seq, d_head]
299         Q = einx.rearrange("... s (h d) -> ... h s d", F.linear(in_features, q_proj_weight), h=num_heads)
300         K = einx.rearrange("... s (h d) -> ... h s d", F.linear(in_features, k_proj_weight), h=num_heads)
301         V = einx.rearrange("... s (h d) -> ... h s d", F.linear(in_features, v_proj_weight), h=num_heads)
302
303         # 3. 注入 RoPE
304         rope = RotaryPositionalEmbedding(theta, d_head, max_seq_len, device=in_features.device)
305         if token_positions is None:
306             token_positions = torch.arange(seq_len, device=in_features.device)
307
308         # 利用 unsqueeze(-2) 配合 RoPE 内部的 einx 逻辑进行 heads 维度的广播
309         pos_for_rope = token_positions.unsqueeze(-2)
310         Q, K = rope(Q, pos_for_rope), rope(K, pos_for_rope)
311
312         # 4. 【核心优化】计算注意力得分 (einx.dot)
313         # 代替 (Q @ K.transpose(-2, -1))
314         # 语义: Query(s_q) 与 Key(s_k) 在特征维 (d) 点积, 保留 batch(...) 和 head(h)
315         logits = einx.dot("... h s_q d, ... h s_k d -> ... h s_q s_k", Q, K) / math.sqrt(d_head)
316
317         # 5. 因果掩码与归一化
318         # 这里的 mask 也可以用 einx 思想理解: 它只作用于最后两个 s 维度
319         mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool, device=in_features.device))
320         # 使用 torch.where 配合布尔掩码比 masked_fill 更符合函数式风格
321         attn_weights = torch.softmax(torch.where(mask, logits, float("-inf")), dim=-1)
322
323         # 6. 【核心优化】加权聚合提取 Value (einx.dot)
324         # 代替 attn_weights @ V
325         # 语义: 用权重 (s_k) 对内容 (s_k) 进行加权求和, 产出查询位置 (s_q) 的特征 (d)
326         out = einx.dot("... h s_q s_k, ... h s_k d -> ... h s_q d", attn_weights, V)
327
328         # 7. 缝合多头并最终投影
329         out = einx.rearrange("... h s d -> ... s (h d)", out)
330         return F.linear(out, o_proj_weight)
331
332
333 def run_rope(
334     d_k: int,
335     theta: float,
336     max_seq_len: int,
337     in_query_or_key: Float[Tensor, "... sequence_length d_k"],
338     token_positions: Int[Tensor, "... sequence_length"],
339 ) -> Float[Tensor, "... sequence_length d_k"]:
340     """
341     Run RoPE for a given input tensor.
342
343     Args:
344         d_k (int): Embedding dimension size for the query or key tensor.
345         theta (float): RoPE parameter.
346         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
347         in_query_or_key (Float[Tensor, "... sequence_length d_k"]): Input tensor to run RoPE on.
348         token_positions (Int[Tensor, "... sequence_length"]): Tensor of shape (batch_size, sequence_length) with
349             ↪ the token positions

```

```

349     Returns:
350         Float[Tensor, " ... sequence_length d_k"]: Tensor with RoPEd input.
351     """
352     # 1. 实例化 RoPE 模块
353     model = RotaryPositionalEmbedding(
354         theta=theta,
355         d_k=d_k,
356         max_seq_len=max_seq_len,
357         device=in_query_or_key.device
358     )
359
360
361     # 2. 评估模式并切断梯度，直接计算
362     model.eval()
363     with torch.no_grad():
364         # 注意要把 token_positions 传进去!
365         output = model(in_query_or_key, token_positions)
366
367     return output
368
369
370 def run_transformer_block(
371     d_model: int,
372     num_heads: int,
373     d_ff: int,
374     max_seq_len: int,
375     theta: float,
376     weights: dict[str, Tensor],
377     in_features: Float[Tensor, " batch sequence_length d_model"],
378 ) -> Float[Tensor, " batch sequence_length d_model"]:
379     """
380     Given the weights of a pre-norm Transformer block and input features,
381     return the output of running the Transformer block on the input features.
382
383     This function should use RoPE.
384     Depending on your implementation, you may simply need to pass the relevant args
385     to your TransformerBlock constructor, or you may need to initialize your own RoPE
386     class and pass that instead.
387
388     Args:
389         d_model (int): The dimensionality of the Transformer block input.
390         num_heads (int): Number of heads to use in multi-headed attention. `d_model` must be
391             evenly divisible by `num_heads`.
392         d_ff (int): Dimensionality of the feed-forward inner layer.
393         max_seq_len (int): Maximum sequence length to pre-cache if your implementation does that.
394         theta (float): RoPE parameter.
395         weights (dict[str, Tensor]):
396             State dict of our reference implementation.
397             The keys of this dictionary are:
398             - `attn.q_proj.weight`
399                 The query projections for all `num_heads` attention heads.
400                 Shape is (d_model, d_model).
401                 The rows are ordered by matrices of shape (num_heads, d_k),
402                 so `attn.q_proj.weight == torch.cat([q_heads.0.weight, ..., q_heads.N.weight], dim=0)`.
403             - `attn.k_proj.weight`
404                 The key projections for all `num_heads` attention heads.
405                 Shape is (d_model, d_model).
406                 The rows are ordered by matrices of shape (num_heads, d_k),
407                 so `attn.k_proj.weight == torch.cat([k_heads.0.weight, ..., k_heads.N.weight], dim=0)`.

```

```

408         - `attn.v_proj.weight`
409             The value projections for all `num_heads` attention heads.
410             Shape is (d_model, d_model).
411             The rows are ordered by matrices of shape (num_heads, d_v),
412             so `attn.v_proj.weight == torch.cat([v_heads.0.weight, ..., v_heads.N.weight], dim=0)`.
413         - `attn.output_proj.weight`
414             Weight of the multi-head self-attention output projection
415             Shape is (d_model, d_model).
416         - `ln1.weight`
417             Weights of affine transform for the first RMSNorm
418             applied in the transformer block.
419             Shape is (d_model,).
420         - `ffn.w1.weight`
421             Weight of the first linear transformation in the FFN.
422             Shape is (d_model, d_ff).
423         - `ffn.w2.weight`
424             Weight of the second linear transformation in the FFN.
425             Shape is (d_ff, d_model).
426         - `ffn.w3.weight`
427             Weight of the third linear transformation in the FFN.
428             Shape is (d_model, d_ff).
429         - `ln2.weight`
430             Weights of affine transform for the second RMSNorm
431             applied in the transformer block.
432             Shape is (d_model,).
433         in_features (Float[Tensor, "batch sequence_length d_model"]):
434             Tensor to run your implementation on.
435
436     Returns:
437         Float[Tensor, "batch sequence_length d_model"] Tensor with the output of
438         running the Transformer block on the input features while using RoPE.
439     """
440     # 1. 实例化
441     block = TransformerBlock(
442         d_model=d_model,
443         num_heads=num_heads,
444         d_ff=d_ff,
445         max_seq_len=max_seq_len,
446         theta=theta,
447         device=in_features.device,
448         dtype=in_features.dtype
449     )
450
451     # 2. 权重映射
452     state_dict = {
453         "ln1.weight": weights["ln1.weight"],
454         "attn.q_proj.W": weights["attn.q_proj.weight"],
455         "attn.k_proj.W": weights["attn.k_proj.weight"],
456         "attn.v_proj.W": weights["attn.v_proj.weight"],
457         "attn.o_proj.W": weights["attn.output_proj.weight"],
458         "ln2.weight": weights["ln2.weight"],
459         "ffn.w1.W": weights["ffn.w1.weight"],
460         "ffn.w2.W": weights["ffn.w2.weight"],
461         "ffn.w3.W": weights["ffn.w3.weight"],
462     }
463
464     # 3. 加载与推理
465     block.load_state_dict(state_dict, strict=True)
466     block.eval()

```

```

467
468     with torch.no_grad():
469         output = block(in_features)
470
471     return output
472
473
474 def run_transformer_lm(
475     vocab_size: int,
476     context_length: int,
477     d_model: int,
478     num_layers: int,
479     num_heads: int,
480     d_ff: int,
481     rope_theta: float,
482     weights: dict[str, Tensor],
483     in_indices: Int[Tensor, " batch_size sequence_length"],
484 ) -> Float[Tensor, " batch_size sequence_length vocab_size"]:
485     r"""Given the weights of a Transformer language model and input indices,
486     return the output of running a forward pass on the input indices.
487
488     This function should use RoPE.
489
490     Args:
491         vocab_size (int): The number of unique items in the output vocabulary to be predicted.
492         context_length (int): The maximum number of tokens to process at once.
493         d_model (int): The dimensionality of the model embeddings and sublayer outputs.
494         num_layers (int): The number of Transformer layers to use.
495         num_heads (int): Number of heads to use in multi-headed attention. `d_model` must be
496             evenly divisible by `num_heads`.
497         d_ff (int): Dimensionality of the feed-forward inner layer (section 3.3).
498         rope_theta (float): The RoPE  $\Theta$  parameter.
499         weights (dict[str, Tensor]):
500             State dict of our reference implementation. {num_layers} refers to an
501             integer between `0` and `num_layers - 1` (the layer index).
502             The keys of this dictionary are:
503             - `token_embeddings.weight`
504               Token embedding matrix. Shape is (vocab_size, d_model).
505             - `layers.{num_layers}.attn.q_proj.weight`
506               The query projections for all `num_heads` attention heads.
507               Shape is (num_heads * (d_model / num_heads), d_model).
508               The rows are ordered by matrices of shape (num_heads, d_k),
509               so `attn.q_proj.weight == torch.cat([q_heads.0.weight, ..., q_heads.N.weight], dim=0)`.
510             - `layers.{num_layers}.attn.k_proj.weight`
511               The key projections for all `num_heads` attention heads.
512               Shape is (num_heads * (d_model / num_heads), d_model).
513               The rows are ordered by matrices of shape (num_heads, d_k),
514               so `attn.k_proj.weight == torch.cat([k_heads.0.weight, ..., k_heads.N.weight], dim=0)`.
515             - `layers.{num_layers}.attn.v_proj.weight`
516               The value projections for all `num_heads` attention heads.
517               Shape is (num_heads * (d_model / num_heads), d_model).
518               The rows are ordered by matrices of shape (num_heads, d_v),
519               so `attn.v_proj.weight == torch.cat([v_heads.0.weight, ..., v_heads.N.weight], dim=0)`.
520             - `layers.{num_layers}.attn.output_proj.weight`
521               Weight of the multi-head self-attention output projection
522               Shape is ((d_model / num_heads) * num_heads, d_model).
523             - `layers.{num_layers}.ln1.weight`
524               Weights of affine transform for the first RMSNorm
525               applied in the transformer block.

```

```

526         Shape is (d_model,).
527     - `layers.{num_layers}.ffn.w1.weight`
528         Weight of the first linear transformation in the FFN.
529         Shape is (d_model, d_ff).
530     - `layers.{num_layers}.ffn.w2.weight`
531         Weight of the second linear transformation in the FFN.
532         Shape is (d_ff, d_model).
533     - `layers.{num_layers}.ffn.w3.weight`
534         Weight of the third linear transformation in the FFN.
535         Shape is (d_model, d_ff).
536     - `layers.{num_layers}.ln2.weight`
537         Weights of affine transform for the second RMSNorm
538         applied in the transformer block.
539         Shape is (d_model,).
540     - `ln_final.weight`
541         Weights of affine transform for RMSNorm applied to the output of the final transformer block.
542         Shape is (d_model, ).
543     - `lm_head.weight`
544         Weights of the language model output embedding.
545         Shape is (vocab_size, d_model).
546     in_indices (Int[Tensor, "batch_size sequence_length"]) Tensor with input indices to run the language model
547     ↪ on. Shape is (batch_size, sequence_length), where
548         `sequence_length` is at most `context_length`.
549
550     Returns:
551         Float[Tensor, "batch_size sequence_length vocab_size"]: Tensor with the predicted unnormalized
552         next-word distribution for each token.
553     """
554
555     # 1. 实例化完整的语言模型
556     model = TransformerLM(
557         vocab_size=vocab_size,
558         context_length=context_length,
559         num_layers=num_layers,
560         d_model=d_model,
561         num_heads=num_heads,
562         d_ff=d_ff,
563         theta=rope_theta, # 把外部的 rope_theta 传给内部需要的 theta
564         device=in_indices.device, # 使用 in_indices 所在的设备
565     )
566
567     # 2. 疯狂的权重重映射 (The Great Weight Remapping)
568     state_dict = {}
569
570     for key, tensor in weights.items():
571         # A. 映射 Embedding (老师的键有 's': token_embeddings)
572         if key == "token_embeddings.weight":
573             state_dict["token_embedding.weight"] = tensor
574
575         # B. 映射最后的 RMSNorm (老师的键叫 ln_final)
576         elif key == "ln_final.weight":
577             state_dict["final_norm.weight"] = tensor
578
579         # C. 映射输出预测头 LM Head
580         elif key == "output.weight":
581             state_dict["lm_head.weight"] = tensor
582
583         # D. 动态映射所有的 Transformer Layers (0, 1, 2... n)
584         elif key.startswith("layers."):
585             parts = key.split(".")

```

```

584         layer_idx = parts[1]
585         sub_key = ".".join(parts[2:])
586
587         if sub_key == "ln1.weight":
588             new_sub_key = "ln1.weight"
589         elif sub_key == "attn.q_proj.weight":
590             new_sub_key = "attn.q_proj.W"
591         elif sub_key == "attn.k_proj.weight":
592             new_sub_key = "attn.k_proj.W"
593         elif sub_key == "attn.v_proj.weight":
594             new_sub_key = "attn.v_proj.W"
595         elif sub_key == "attn.output_proj.weight":
596             new_sub_key = "attn.o_proj.W"
597         elif sub_key == "ln2.weight":
598             new_sub_key = "ln2.weight"
599         elif sub_key == "ffn.w1.weight":
600             new_sub_key = "ffn.w1.W"
601         elif sub_key == "ffn.w2.weight":
602             new_sub_key = "ffn.w2.W"
603         elif sub_key == "ffn.w3.weight":
604             new_sub_key = "ffn.w3.W"
605         else:
606             new_sub_key = sub_key
607
608         state_dict[f"layers.{layer_idx}.{new_sub_key}"] = tensor
609
610         # E. 兜底策略
611     else:
612         state_dict[key] = tensor
613
614     # 3. 严格加载映射好的权重
615     model.load_state_dict(state_dict, strict=True)
616
617     # 4. 执行神圣的前向推理
618     model.eval()
619     with torch.no_grad():
620         # 注意: 这里传进去的参数变成了 in_indices
621         output = model(in_indices)
622
623     return output
624
625
626 def run_rmsnorm(
627     d_model: int,
628     eps: float,
629     weights: Float[Tensor, " d_model"],
630     in_features: Float[Tensor, " ... d_model"],
631 ) -> Float[Tensor, " ... d_model"]:
632     """Given the weights of a RMSNorm affine transform,
633     return the output of running RMSNorm on the input features.
634
635     Args:
636         d_model (int): The dimensionality of the RMSNorm input.
637         eps (float): A value added to the denominator for numerical stability.
638         weights (Float[Tensor, "d_model"]): RMSNorm weights.
639         in_features (Float[Tensor, "... d_model"]): Input features to run RMSNorm on. Can have arbitrary leading
640             dimensions.
641
642     Returns:

```

```

643         Float[Tensor, "... d_model"]: Tensor of with the same shape as `in_features` with the output of running
644         RMSNorm of the `in_features`.
645     """
646     model = RMSNorm(
647         d_model=d_model,
648         eps=eps,
649         device=weights.device,
650         dtype=weights.dtype
651     )
652
653     model.load_state_dict({"weight": weights}, strict=True)
654
655     model.eval()
656     with torch.no_grad():
657         output = model(in_features)
658
659     return output
660
661
662 def run_silu(in_features: Float[Tensor, "..."]) -> Float[Tensor, "..."]:
663     """Given a tensor of inputs, return the output of applying SiLU
664     to each element.
665
666     Args:
667         in_features(Float[Tensor, "...]): Input features to run SiLU on. Shape is arbitrary.
668
669     Returns:
670         Float[Tensor, "..."]: of with the same shape as `in_features` with the output of applying
671         SiLU to each element.
672     """
673     raise NotImplementedError
674
675
676 def run_get_batch(
677     dataset: npt.NDArray, batch_size: int, context_length: int, device: str
678 ) -> tuple[torch.Tensor, torch.Tensor]:
679     """
680     Given a dataset (a 1D numpy array of integers) and a desired batch size and
681     context length, sample language modeling input sequences and their corresponding
682     labels from the dataset.
683
684     Args:
685         dataset (np.array): 1D numpy array of integer token IDs in the dataset.
686         batch_size (int): Desired batch size to sample.
687         context_length (int): Desired context length of each sampled example.
688         device (str): PyTorch device string (e.g., 'cpu' or 'cuda:0') indicating the device
689         to place the sampled input sequences and labels on.
690
691     Returns:
692         Tuple of torch.LongTensors of shape (batch_size, context_length). The first tuple item
693         is the sampled input sequences, and the second tuple item is the corresponding
694         language modeling labels.
695     """
696     raise NotImplementedError
697
698
699 def run_softmax(in_features: Float[Tensor, "..."], dim: int) -> Float[Tensor, "..."]:
700     """
701     Given a tensor of inputs, return the output of softmaxing the given `dim`

```

```

702     of the input.
703
704     Args:
705         in_features (Float[Tensor, "..."]): Input features to softmax. Shape is arbitrary.
706         dim (int): Dimension of the `in_features` to apply softmax to.
707
708     Returns:
709         Float[Tensor, "..."]: Tensor of with the same shape as `in_features` with the output of
710         softmax normalizing the specified `dim`.
711     """
712     # 1. 寻找指定维度上的最大值
713     # keepdim=True 是灵魂! 它能保留维度形状 (比如把形状从 [3, 4] 变成 [3, 1]),
714     # 这样后续减法时才能正确触发广播机制 (Broadcasting)。
715     # 注意: torch.max 会返回一个元组 (values, indices), 我们只需要取第 0 项的 values。
716     max_vals = in_features.max(dim=dim, keepdim=True)[0]
717
718     # 2. Max Trick: 平移输入值, 防止指数爆炸
719     shifted_logits = in_features - max_vals
720
721     # 3. 计算指数 (现在所有的输入都 <= 0, 所以 exp 的结果都在 0~1 之间, 绝对安全)
722     exps = torch.exp(shifted_logits)
723
724     # 4. 沿着指定维度求和, 作为分母
725     sum_exps = exps.sum(dim=dim, keepdim=True)
726
727     # 5. 归一化, 得到最终的概率分布
728     probabilities = exps / sum_exps
729
730     return probabilities
731
732
733 def run_cross_entropy(
734     inputs: Float[Tensor, "batch_size vocab_size"], targets: Int[Tensor, "batch_size"]
735 ) -> Float[Tensor, ""]:
736     """Given a tensor of inputs and targets, compute the average cross-entropy
737     loss across examples.
738
739     Args:
740         inputs (Float[Tensor, "batch_size vocab_size"]): inputs[i][j] is the
741         unnormalized logit of jth class for the ith example.
742         targets (Int[Tensor, "batch_size"]): Tensor of shape (batch_size,) with the index of the correct class.
743         Each value must be between 0 and `num_classes - 1`.
744
745     Returns:
746         Float[Tensor, ""]: The average cross-entropy loss across examples.
747     """
748     raise NotImplementedError
749
750
751 def run_gradient_clipping(parameters: Iterable[torch.nn.Parameter], max_l2_norm: float) -> None:
752     """Given a set of parameters, clip their combined gradients to have l2 norm at most max_l2_norm.
753
754     Args:
755         parameters (Iterable[torch.nn.Parameter]): collection of trainable parameters.
756         max_l2_norm (float): a positive value containing the maximum l2-norm.
757
758     The gradients of the parameters (parameter.grad) should be modified in-place.
759     """
760     raise NotImplementedError

```



```

761
762
763 def get_adamw_cls() -> Any:
764     """
765     Returns a torch.optim.Optimizer that implements AdamW.
766     """
767     raise NotImplementedError
768
769
770 def run_get_lr_cosine_schedule(
771     it: int,
772     max_learning_rate: float,
773     min_learning_rate: float,
774     warmup_iters: int,
775     cosine_cycle_iters: int,
776 ):
777     """
778     Given the parameters of a cosine learning rate decay schedule (with linear
779     warmup) and an iteration number, return the learning rate at the given
780     iteration under the specified schedule.
781
782     Args:
783         it (int): Iteration number to get learning rate for.
784         max_learning_rate (float): alpha_max, the maximum learning rate for
785             cosine learning rate schedule (with warmup).
786         min_learning_rate (float): alpha_min, the minimum / final learning rate for
787             the cosine learning rate schedule (with warmup).
788         warmup_iters (int): T_w, the number of iterations to linearly warm-up
789             the learning rate.
790         cosine_cycle_iters (int): T_c, the number of cosine annealing iterations.
791
792     Returns:
793         Learning rate at the given iteration under the specified schedule.
794     """
795     raise NotImplementedError
796
797
798 def run_save_checkpoint(
799     model: torch.nn.Module,
800     optimizer: torch.optim.Optimizer,
801     iteration: int,
802     out: str | os.PathLike | BinaryIO | IO[bytes],
803 ):
804     """
805     Given a model, optimizer, and an iteration number, serialize them to disk.
806
807     Args:
808         model (torch.nn.Module): Serialize the state of this model.
809         optimizer (torch.optim.Optimizer): Serialize the state of this optimizer.
810         iteration (int): Serialize this value, which represents the number of training iterations
811             we've completed.
812         out (str | os.PathLike | BinaryIO | IO[bytes]): Path or file-like object to serialize the model,
813             ↪ optimizer, and iteration to.
814     """
815     raise NotImplementedError
816
817 def run_load_checkpoint(
818     src: str | os.PathLike | BinaryIO | IO[bytes],

```

```

819     model: torch.nn.Module,
820     optimizer: torch.optim.Optimizer,
821 ) -> int:
822     """
823     Given a serialized checkpoint (path or file-like object), restore the
824     serialized state to the given model and optimizer.
825     Return the number of iterations that we previously serialized in
826     the checkpoint.
827
828     Args:
829         src (str | os.PathLike | BinaryIO | IO[bytes]): Path or file-like object to serialized checkpoint.
830         model (torch.nn.Module): Restore the state of this model.
831         optimizer (torch.optim.Optimizer): Restore the state of this optimizer.
832     Returns:
833         int: the previously-serialized number of iterations.
834     """
835     raise NotImplementedError
836
837
838 def get_tokenizer(
839     vocab: dict[int, bytes],
840     merges: list[tuple[bytes, bytes]],
841     special_tokens: list[str] | None = None,
842 ) -> Any:
843     """Given a vocabulary, a list of merges, and a list of special tokens,
844     return a BPE tokenizer that uses the provided vocab, merges, and special tokens.
845
846     Args:
847         vocab (dict[int, bytes]): The tokenizer vocabulary, a mapping from int (token ID in the vocabulary)
848             to bytes (token bytes)
849         merges (list[tuple[bytes, bytes]]): BPE merges. Each list item is a tuple of bytes (<token1>, <token2>),
850             representing that <token1> was merged with <token2>.
851             Merges are ordered by order of creation.
852         special_tokens (list[str] | None): A list of string special tokens for the tokenizer. These strings will
853             ↪ never
854             be split into multiple tokens, and will always be kept as a single token.
855
856     Returns:
857         A BPE tokenizer that uses the provided vocab, merges, and special tokens.
858     """
859     return Tokenizer(vocab=vocab, merges=merges, special_tokens=special_tokens)
860
861 def run_train_bpe(
862     input_path: str,
863     vocab_size: int,
864     special_tokens: list[str]
865 ) -> tuple[dict[int, bytes], list[tuple[bytes, bytes]]]:
866     """
867     Adapter function for training the BPE tokenizer.
868     This routes the inputs from the test suite to our optimized implementation.
869     """
870
871     # 直接调用核心逻辑并返回结果
872     vocab, merges = train_bpe(
873         input_path=input_path,
874         vocab_size=vocab_size,
875         special_tokens=special_tokens
876     )

```

```
877  
878     return vocab, merges
```
